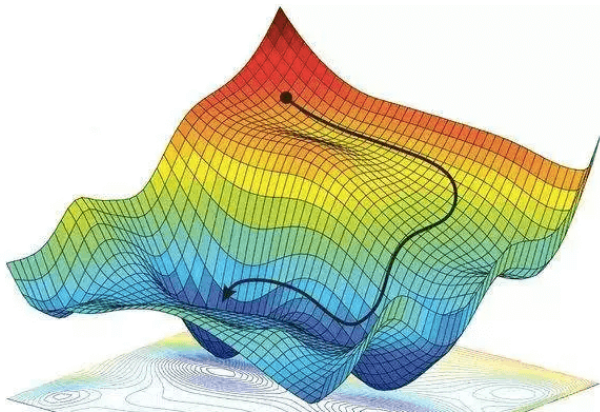


Deep Learning

Training DNNs - I

Loss surface



Source: Medium

Optimization

- Finding the parameters that minimize the training loss

$$W^*, \mathbf{b}^* = \operatorname{argmin}_{W, \mathbf{b}} \mathcal{L}(f(\cdot; W, \mathbf{b}); \mathcal{D})$$

Optimization

- Finding the parameters that minimize the training loss

$$W^*, \mathbf{b}^* = \operatorname{argmin}_{W, \mathbf{b}} \mathcal{L}(f(\cdot; W, \mathbf{b}); \mathcal{D})$$

- How do we find these optimal parameters?

Optimization

- Finding the parameters that minimize the training loss

$$W^*, \mathbf{b}^* = \underset{W, \mathbf{b}}{\operatorname{argmin}} \mathcal{L}(f(\cdot; W, \mathbf{b}); \mathcal{D})$$

- How do we find these optimal parameters?
 - Closed form solution (e.g. linear regression)

Optimization

- Finding the parameters that minimize the training loss

$$W^*, \mathbf{b}^* = \underset{W, \mathbf{b}}{\operatorname{argmin}} \mathcal{L}(f(\cdot; W, \mathbf{b}); \mathcal{D})$$

- How do we find these optimal parameters?
 - Closed form solution (e.g. linear regression)
 - What if the loss function can't be minimized analytically?

Optimization

- Finding the parameters that minimize the training loss

$$W^*, \mathbf{b}^* = \underset{W, \mathbf{b}}{\operatorname{argmin}} \mathcal{L}(f(\cdot; W, \mathbf{b}); \mathcal{D})$$

- How do we find these optimal parameters?
 - Closed form solution (e.g. linear regression)
 - What if the loss function can't be minimized analytically?
 - Gradient Descent with Back propagation

Batch Gradient Descent

```
for i in range(nb_epochs):  
     $\nabla L_w$  = evaluate_gradient(L,  $\mathcal{D}$ , w)  
    w = w -  $\eta$  *  $\nabla L_w$ 
```


Batch Gradient Descent

```
for i in range(nb_epochs):  
     $\nabla L_w$  = evaluate_gradient(L,  $\mathcal{D}$ , w)  
    w = w -  $\eta$  *  $\nabla L_w$ 
```

- ① Guaranteed to converge to global minima in case of convex functions, and to a local minima in case of non-convex functions

Stochastic Gradient Descent (SGD)

- ① Performs updates parameters for each training example

$$w = w - \eta \nabla_w \mathcal{L}(w, x^i, y^i)$$

Stochastic Gradient Descent (SGD)

- ① Performs updates parameters for each training example

$$w = w - \eta \nabla_w \mathcal{L}(w, x^i, y^i)$$

- ② In case of large datasets, Batch GD computes redundant gradients for similar examples for each parameter update

Stochastic Gradient Descent (SGD)

- ① Performs updates parameters for each training example
$$w = w - \eta \nabla_w \mathcal{L}(w, x^i, y^i)$$
- ② In case of large datasets, Batch GD computes redundant gradients for similar examples for each parameter update
- ③ SGD does away with redundancy and generally faster and can be used to learn online

SGD: Computes the gradient on a single example (or a small batch). This estimate is noisy. Even if the true gradient is zero (a saddle point), the noisy approximation will likely point in a random direction, allowing it to escape from saddle point.

Computational Speed & Convergence Time : SGD updates weights after seeing 1 examples and model starts learning immediately.

Memory efficiency : SGD requires holding only one sample or mini batch in memory.

Stochastic Gradient Descent (SGD)

- ① However, frequent updates with a high variance cause the objective function to fluctuate heavily

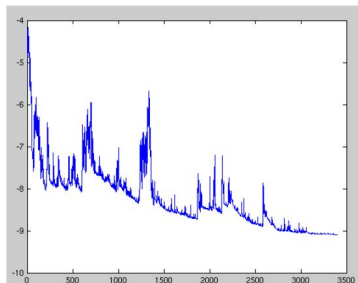


Figure credits: Wikipedia

Stochastic Gradient Descent (SGD)

- ① SGD's fluctuations enable it to jump to new and potentially better local minima

Stochastic Gradient Descent (SGD)

- ① SGD's fluctuations enable it to jump to new and potentially better local minima
- ② This complicates the convergence, as it overshoots

Stochastic Gradient Descent (SGD)

- ① SGD's fluctuations enable it to jump to new and potentially better local minima
- ② This complicates the convergence, as it overshoots
- ③ However, if the learning rate is slowly decreased, we can show similar convergence to Batch GD

Stochastic Gradient Descent (SGD)

```
for i in range(nb_epochs):  
    np.random.shuffle( $\mathcal{D}$ )  
    for  $x_i \in \mathcal{D}$ :  
         $\nabla L_w = \text{evaluate\_gradient}(L, x_i, w)$   
         $w = w - \eta * \nabla L_w$ 
```

Mini-batch Gradient Descent

- ① Takes the best of both worlds, updates the parameters for every mini-batch of n samples

$$w = w - \eta \nabla_w \mathcal{L}(w, x^{i:i+n}, y^{i:i+n})$$

Mini-batch Gradient Descent

- ① Takes the best of both worlds, updates the parameters for every mini-batch of n samples

$$w = w - \eta \nabla_w \mathcal{L}(w, x^{i:i+n}, y^{i:i+n})$$

- ②
 - Reduces the variance of the parameter updates, which can lead to more stable convergence
 - Can make use of highly optimized matrix optimizations

Mini-batch Gradient Descent

- ① Takes the best of both worlds, updates the parameters for every mini-batch of n samples

$$w = w - \eta \nabla_w \mathcal{L}(w, x^{i:i+n}, y^{i:i+n})$$

- ②
 - Reduces the variance of the parameter updates, which can lead to more stable convergence
 - Can make use of highly optimized matrix optimizations
- ③ Common mini-batch sizes vary from 32 to 1024, depending on the application

Mini-batch Gradient Descent

- ① Takes the best of both worlds, updates the parameters for every mini-batch of n samples
$$w = w - \eta \nabla_w \mathcal{L}(w, x^{i:i+n}, y^{i:i+n})$$
- ②
 - Reduces the variance of the parameter updates, which can lead to more stable convergence
 - Can make use of highly optimized matrix optimizations
- ③ Common mini-batch sizes vary from 32 to 1024, depending on the application
- ④ This is the algorithm of choice while training DNNs (also, incorrectly referred to as SGD in general)

Mini-batch Gradient Descent

```
for i in range(nb_epochs):  
    np.random.shuffle( $\mathcal{D}$ )  
    for batch in get_batches( $\mathcal{D}$ , batch_size = 128):  
         $\nabla L_w$  = evaluate_gradient(L, batch, w)  
         $w = w - \eta * \nabla L_w$ 
```

Some challenges

- ① Choosing a proper learning rate

Some challenges

- ① Choosing a proper learning rate
 - Learning rate schedules try to adjust it during the training

Some challenges

- ① Choosing a proper learning rate
 - Learning rate schedules try to adjust it during the training
 - However, these schedules are defined in advance and hence unable to adapt to the task at hand

Some challenges

- ① Choosing a proper learning rate
 - Learning rate schedules try to adjust it during the training
 - However, these schedules are defined in advance and hence unable to adapt to the task at hand
- ② Same learning rate applies to all the parameters

Some challenges

- ① Choosing a proper learning rate
 - Learning rate schedules try to adjust it during the training
 - However, these schedules are defined in advance and hence unable to adapt to the task at hand
- ② Same learning rate applies to all the parameters
- ③ Avoiding numerous sub-optimal local minima

Issues with SGD

- DNNs are trained via SGD: $w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$

Issues with SGD

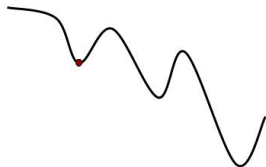
- DNNs are trained via SGD: $w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$
- Loss is a high dimensional function

Issues with SGD

- DNNs are trained via SGD: $w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$
- Loss is a high dimensional function
 - May have local minima

Issues with SGD

- DNNs are trained via SGD: $w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$
- Loss is a high dimensional function
 - May have local minima
 - May have saddle points



Stuck at a local minimum

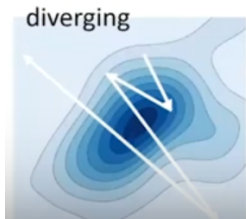


Stuck at a saddle point

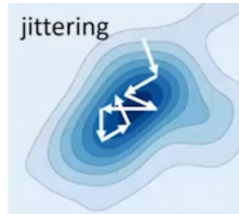
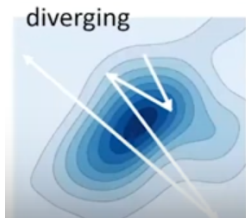
Convergence of Gradient Descent



Convergence of Gradient Descent



Convergence of Gradient Descent



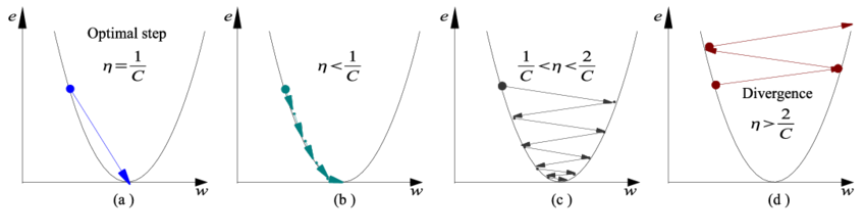
Convergence of Gradient Descent

- When does it diverge?

Convergence of Gradient Descent

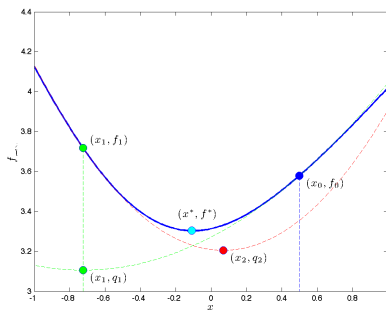
- When does it diverge?
- How to ensure smooth convergence? (Conditions for convergence)

Convergence for Quadratic functions



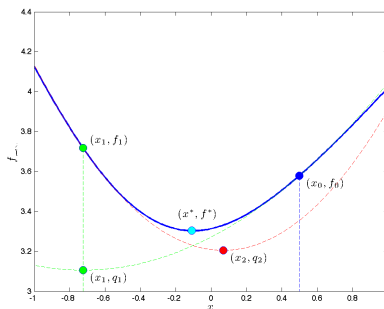
In case of generic and convex functions

- Perform a quadratic approximation



In case of generic and convex functions

- Perform a quadratic approximation
- $\eta_{opt} = \frac{1}{f''}$ (Newton's Method)



Multivariate functions

- $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} + \mathbf{x}^T\mathbf{b} + \mathbf{c}$

Multivariate functions

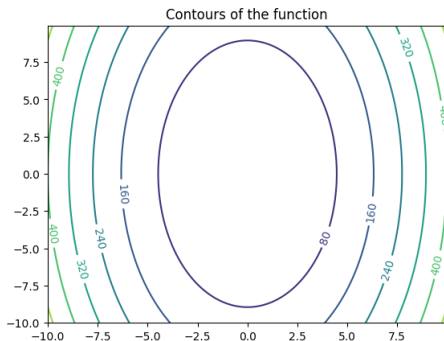
- $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} + \mathbf{x}^T\mathbf{b} + \mathbf{c}$
- For convex functions, A is positive definite

Multivariate functions

- $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{b} + \mathbf{c}$
- For convex functions, A is positive definite
- (For simplicity) If A is diagonal (+ve entries for convex f), then f is sum of multiple quadratic functions

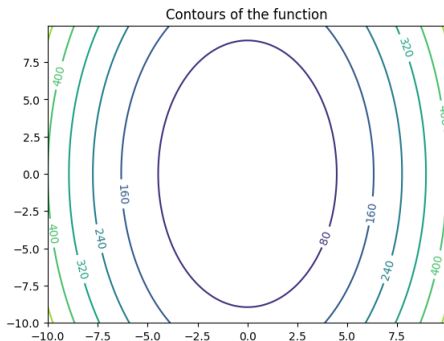
Multivariate functions

- Optimization gets decoupled (each component can be optimized independently)



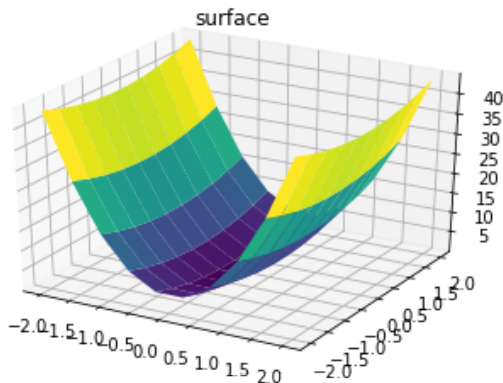
Multivariate functions

- Optimization gets decoupled (each component can be optimized independently)
- Optimal Learning rate is different for different components



Issues with SGD

- DNNs are trained via SGD: $w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$
- Loss is a high dimensional function
 - May vary swiftly in one direction and slowly in the other



Issues with SGD

- Learning rate must be smaller than the twice the smallest optimal learning rate $\eta < 2 \cdot \eta_{min}$

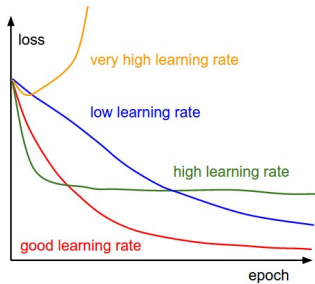
Issues with SGD

- Learning rate must be smaller than the twice the smallest optimal learning rate $\eta < 2 \cdot \eta_{min}$
- Else, it may diverge

Issues with SGD

- Learning rate must be smaller than the twice the smallest optimal learning rate $\eta < 2 \cdot \eta_{min}$
- Else, it may diverge
- This makes the convergence slow (and oscillate in some directions)

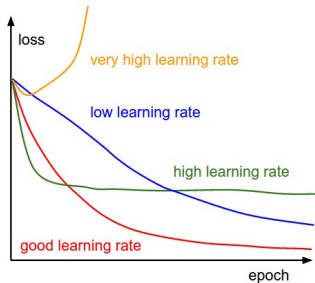
Learning rate (lr)



- What lr to use?

Figure credits: CS231n-Stanford

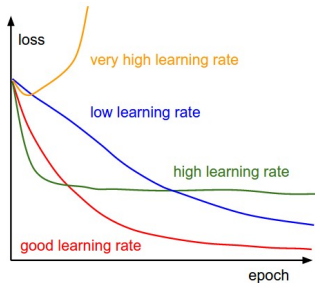
Learning rate (lr)



- What lr to use?
- Different lr at different stages of the training!

Figure credits: CS231n-Stanford

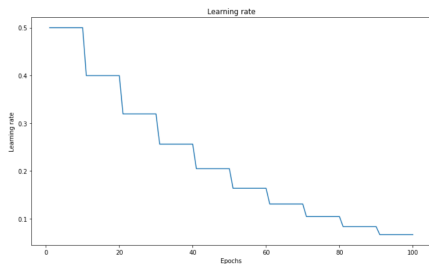
Learning rate (lr)



- What lr to use?
- Different lr at different stages of the training!
- Start with high lr and reduce it with time

Figure credits: CS231n-Stanford

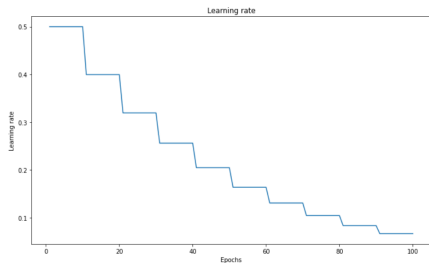
Learning Rate decay: Step



- 1 Reduce the lr after regular intervals

Figure credits: Katherine Li

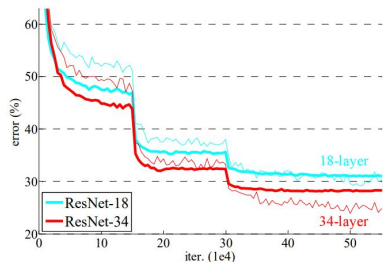
Learning Rate decay: Step



- 1 Reduce the lr after regular intervals
- 2 E.g. after every 30 epochs, $\eta^* = 0.1 \cdot \eta$

Figure credits: Katherine Li

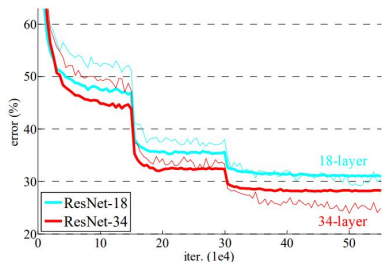
Learning Rate decay: Step



- ① Characteristic loss curve: different phases for 'stage'

Figure credits: Kaiming He et al. 2015, ResNets

Learning Rate decay: Step



- ① Characteristic loss curve: different phases for "stage"
- ② Issues: annoying hyper-params (when to reduce, by how much, etc.)

Figure credits: Kaiming He et al. 2015, ResNets

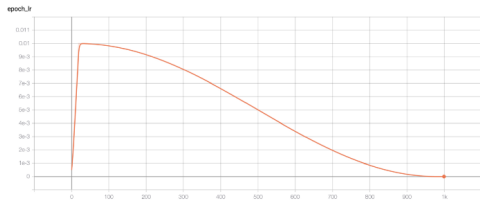
Learning Rate decay: Cosine



- ① Reduces the lr continuously
- $$\eta_t = \frac{1}{2}\eta_0(1 + \cos(t\pi/T))$$

Figure credits: Sebastian Correa and Medium.com

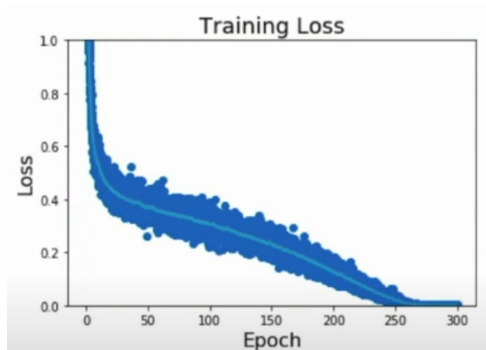
Learning Rate decay: Cosine



- 1 Reduces the lr continuously
$$\eta_t = \frac{1}{2}\eta_0(1 + \cos(t\pi/T))$$
- 2 Less number of hyper-parameters

Figure credits: [Sebastian Correa](#) and [Medium.com](#)

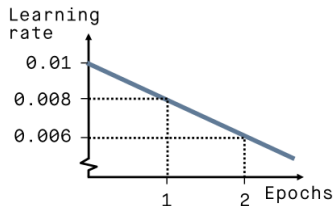
Learning Rate decay: Cosine



- ① Training longer tends to work, but initial lr is still a tricky one

Figure credits: Dr Justin Johnson, U Michigan

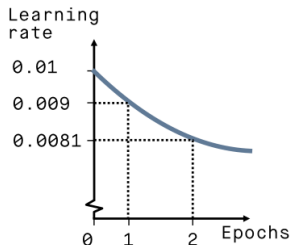
Learning Rate decay: Linear



$$\textcircled{1} \quad \eta_t = \eta_0(1 - t/T)$$

Figure credits: peltarion.com

Learning Rate decay: Exponential



$$\textcircled{1} \quad \eta_t = \eta_0 \cdot (1 - \alpha/100)^t$$

Learning Rate decay: Constant lr

- ① No change in the learning rate

$$\eta_t = \eta_0$$

Learning Rate decay: Constant lr

- ① No change in the learning rate

$$\eta_t = \eta_0$$

- ② Works for prototyping of ideas (other schedules may be better for squeezing in those 1-2% of gains in the performance)

Issues with SGD

- SGD leads to jitter along the deep dimension and slow progress along the shallow one

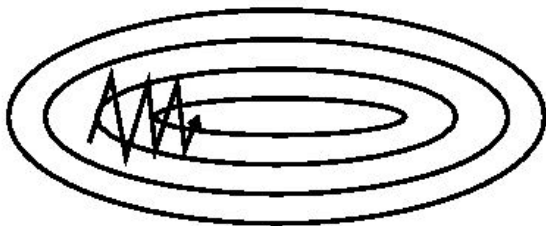


Figure credits: Sebastian Ruder

SGD+Momentum

SGD+Momentum

SGD

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

$$v_0 = 0$$

$$v_{t+1} = \rho \cdot v_t + \nabla_w J(w)$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1}$$

SGD+Momentum

SGD+Momentum

SGD

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

$$v_0 = 0$$

$$v_{t+1} = \rho \cdot v_t + \nabla_w J(w)$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1}$$

- Aggregates velocity: exponential moving average over gradients

SGD+Momentum

SGD+Momentum

SGD

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

$$v_0 = 0$$

$$v_{t+1} = \rho \cdot v_t + \nabla_w J(w)$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1}$$

- Aggregates velocity: exponential moving average over gradients
- ρ is the friction (typically set to 0.9 or 0.99)

SGD+Momentum

SGD

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

```
for i in range(num_iters):  
    → dw = grad(J, W, x, y)  
    → w ← w - η · dw
```

SGD+Momentum

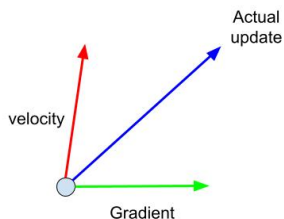
$$v_0 = 0$$

$$v_{t+1} = \rho \cdot v_t + \nabla_w J(w)$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1}$$

```
v_0 = 0  
for i in range(num_iters):  
    → dw = grad(J, W, x, y)  
    → v = ρ · v + dw  
    → w ← w - η · v
```

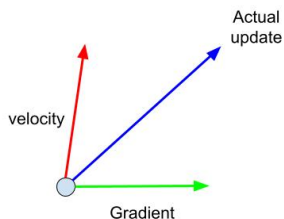
SGD+Momentum



Momentum Update

① How can momentum help?

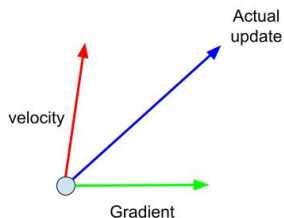
SGD+Momentum



Momentum Update

- ① How can momentum help?
 - Optimization proceeds even at the local minimum or saddle point (because of the accumulated velocity)

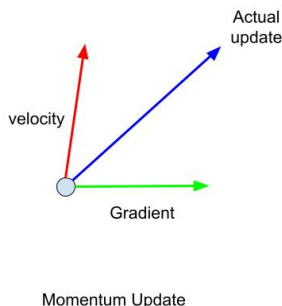
SGD+Momentum



Momentum Update

- ① How can momentum help?
- Optimization proceeds even at the local minimum or saddle point (because of the accumulated velocity)
 - Jitter is reduced in ravine like loss surfaces

SGD+Momentum

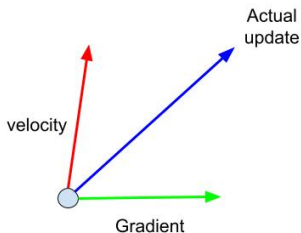


① How can momentum help?

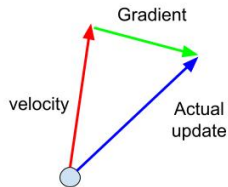
- Optimization proceeds even at the local minimum or saddle point (because of the accumulated velocity)
- Jitter is reduced in ravine like loss surfaces
- Updates are more smoothed out (less noisy because of the exponential averaging)

Nesterov Momentum

- ① Look ahead with the velocity, then take a step in the gradient's direction



Momentum Update



Nesterov Momentum

Nesterov Momentum

```
 $v_0 = 0$   
for i in range(num_iters):  
     $\rightarrow dw = \text{grad}(J, W + \rho \cdot v, x, y)$   
     $\rightarrow v = \rho \cdot v + dw$   
     $\rightarrow w = w - \eta \cdot v$ 
```

NAG allows to change velocity in a faster and more responsive way (particularly for large values of ρ)

Ada Grad

- ① Goal: Adaptive (or, per-parameter) learning rates are introduced

Ada Grad

- ① Goal: Adaptive (or, per-parameter) learning rates are introduced
- ② Parameter-wise scaling of the learning rate by the aggregated gradient

Ada Grad

```
grad_sq = 0
for i in range(max_iters):
    → dw = grad(J,w,x,y)
    → grad_sq += dw ⊙ dw
    →  $w- = \eta \cdot dw / (\text{sqrt}(\text{grad\_sq}) + \epsilon)$ 
```

Ada Grad

```
grad_sq = 0
for i in range(max_iters):
    → dw = grad(J,w,x,y)
    → grad_sq += dw ⊙ dw
    →  $w ← w - \eta \cdot dw / (\text{sqrt}(\text{grad\_sq}) + \epsilon)$ 
```

- Smaller (**larger**) updates for parameters associated with frequently (**infrequently**) occurring features

Ada Grad

```
grad_sq = 0
for i in range(max_iters):
    → dw = grad(J,w,x,y)
    → grad_sq += dw ⊙ dw
    →  $w- = \eta \cdot dw / (\text{sqrt}(\text{grad\_sq}) + \epsilon)$ 
```

- Smaller (**larger**) updates for parameters associated with frequently (**infrequently**) occurring features
- well-suited for dealing with sparse data

RMS Prop

- ① If Ada Grad is run for too long
 - the gradients accumulate to a big value
 - $\rightarrow$ update becomes too small (or, learning rate is reduced continuously)

RMS Prop

- ① If Ada Grad is run for too long
 - the gradients accumulate to a big value
 - $\rightarrow$ update becomes too small (or, learning rate is reduced continuously)
- ② RMS prop (a leaky version of Ada Grad) addresses this using a friction coefficient (ρ)

RMS Prop

```
grad_sq = 0
for i in range(max_iters):
    → dw = grad(J,w,x,y)
    → grad_sq =  $\rho \cdot \text{grad\_sq} + (1 - \rho) \cdot dw \odot dw$ 
    →  $w- = \eta \cdot dw / (\text{sqrt}(\text{grad\_sq}) + \epsilon)$ 
```

Adam

- ① Inculcates both the good things: momentum and the adaptive learning rates

Adam = RMSProp + Momentum

Adam

- ① Inculcates both the good things: momentum and the adaptive learning rates

Adam = RMSProp + Momentum

- ② $m1 = 0$

$m2 = 0$

for i in range(max_iters):

→ $dw = \text{grad}(J, w, x, y)$

→ $m1 = \beta_1 \cdot m1 + (1 - \beta_1) \cdot dw$

→ $m2 = \beta_2 \cdot m2 + (1 - \beta_2) \cdot dw^2$

→ $w- = \eta \cdot m1 / (\text{sqrt}(m2) + \epsilon)$

Adam

```
①  $m1 = 0$   
 $m2 = 0$   
for i in range(max_iters):  
    →  $dw = \text{grad}(J, w, x, y)$   
    →  $m1 = \beta_1 \cdot m1 + (1 - \beta_1) \cdot dw$   
    →  $m2 = \beta_2 \cdot m2 + (1 - \beta_2) \cdot dw^2$   
    →  $w- = \eta \cdot m1 / (\text{sqrt}(m2) + \epsilon)$ 
```

Adam

- ① $m1 = 0$
 $m2 = 0$
for i in range(max_iters):
→ $dw = \text{grad}(J, w, x, y)$
→ $m1 = \beta_1 \cdot m1 + (1 - \beta_1) \cdot dw$
→ $m2 = \beta_2 \cdot m2 + (1 - \beta_2) \cdot dw^2$
→ $w- = \eta \cdot m1 / (\text{sqrt}(m2) + \epsilon)$
- ② Bias correction is performed (since the estimates start from 0)

Adam

- ① $m1 = 0$
 $m2 = 0$
for i in range(max_iters):
→ $dw = \text{grad}(J, w, x, y)$
→ $m1 = \beta_1 \cdot m1 + (1 - \beta_1) \cdot dw$
→ $m2 = \beta_2 \cdot m2 + (1 - \beta_2) \cdot dw^2$
→ $w- = \eta \cdot m1 / (\text{sqrt}(m2) + \epsilon)$
- ② Bias correction is performed (since the estimates start from 0)
- ③ Adam works well in practice (mostly with a fixed set of values for the hyper-params)

Many more variants exist

- ① AMSGrad
- ② Nadam
- ③ AdaMax
- ④ AdaDelta